- In linear regression, assuming Gaussian noise leads to L2 loss, because maximizing Gaussian likelihood is equivalent to minimizing squared error.
- Ridge regression corresponds to MAP estimation with Gaussian prior on the weights.
- Cross validation is commonly used to choose hyperparameters and estimate test error
- in ridge, as $\lambda \uparrow$, typically bias ↑ and variance ↓
- L_0: stepwise search (counts features), L_1: sparse solutions, L_2: Shrinks weights but none to 0.
- the softmax function maps a score vector to a probability distribution
- A standard way of picking RBF centers is to run k-means and use the cluster centers
- Gaussian RBFs are typically local. ? polynomial basis functions are global ?
- If an event is certain, the entropy $H(X) = 0$
- In NNs, max pooling reduces computation and adds some translation invariance
- PCA minimizes the reconstruction error!
- the distributional similarity hypothesis used for word embeddings states: "words occurring in similar contexts tend to have similar meanings"
- Autoencoders learn to reconstruct inputs through an encoder-decoder.
- A linear autoencoder approximates PCA
- A denoising autoencoder is trained to reconstruct a clean signal from a corrupted input
- k means minimizes the within-cluster squared reconstruction error
- Naive Bayes assumes that words are indep given the topic / class
- If you don't observe the topic in Naive Bayes, you can use MLE instead
- Two sets of nodes X and Y are d-separated by a set of nodes Z if X and Y are conditionally independent given Z: $X \perp Y | Z$
- L_0 & $L_{1/2}$ are not convex, L_1 & L_2 are convex.
- k means assumes all clusters are roughly the same size (inductive bias)
- CNN inductive biases: locality, translational invariance (

Rh: • HDPs generalize Markov Models. PO HDPs generalize HHMs. "HDPs generalize to NNets"

"In the limit, ridge reg shrinks the params to 0 hence 0 variance ↑ "OLS linear regression is unbiased in $\hat{\theta}$ or $\hat{y}$, ridge makes it biased ↑

How to solve

$p=2$: $(X^TX + \lambda I)^{-1} X^T y$

$p=1$: Gradient descent

$p=0$ search (stepwise or streamwise)

L_2 regression = $\arg\min_w \|y - Xw\|_2$ = OLS ?

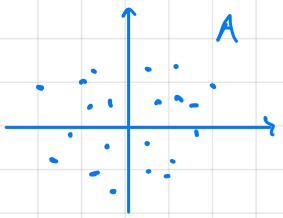
L_1 = " $_1$

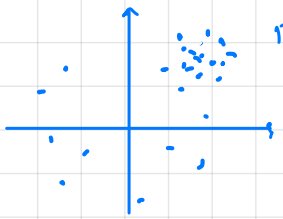
penalty most heavily shrinks large weights

- L_0 penalty most strongly encourages weights to be 0
- L_0 penalty is scale invariant, L_1 & L_2 are not

Elastic net, convex, = ridge + Lasso

kernel regression v/s k-nn:





If we have case B, we rather use k-nn

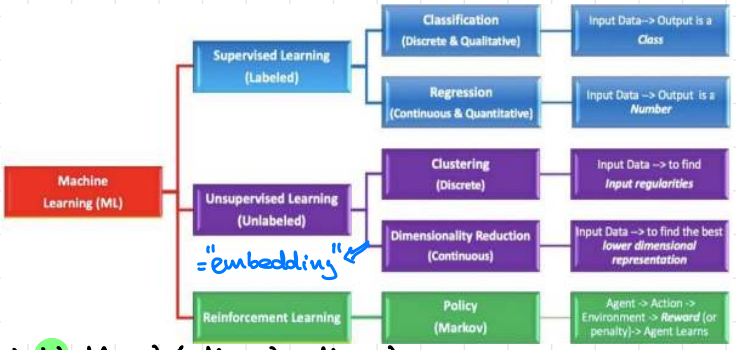
If we have case A, we rather use kernel regression!

relationship kmeans knn.

1a)

ML: i) components: Rep, Loss, Opt ; ii) types: SL, UL, RL ;

iii) models: generative $p(x)$ / discriminative $p(y|x)$ { parametric / non-parametric / semi-parametric } knn, decision trees, kernel regression



1b)

k-NN can do both regression & classification: $k\uparrow \Rightarrow$ model complexity $\downarrow$

non-parametric but has two hyperparameters: k and $d(\cdot,\cdot)$ (distance metric)

algo: 1) find k nearest neighbors according to $d(\cdot,\cdot)$

2) $\hat{y}(x)$ = majority label or average y's of those k neighbors

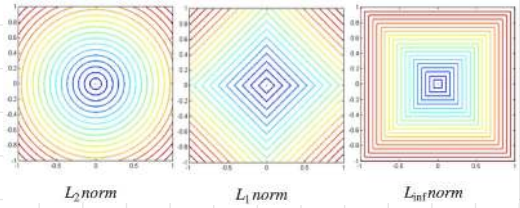
distance properties: i) $d(x,y) \geq 0$; ii) $d(x,y) = 0 \Leftrightarrow x = y$; iii) $d(x,y) = d(y,x)$; iv) $d(x,z) \leq d(x,y) + d(y,z)$

L_p norm: $\|\underline{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}}$ → props: i) $\|\underline{x}\| = 0 \Leftrightarrow \underline{x}$ is the 0 vector ii) $\|a\,\underline{x}\|_p = |a|\,\|\underline{x}\|_p$ (homogeneity)

iii) $\|\underline{u}+\underline{v}\|_p \leq \|\underline{u}\|_p + \|\underline{v}\|_p$ (triangle ineq or subadditivity) iv) $p \geq 1$?

L_0 pseudo-norm $= \|\underline{x}\|_0 =$ the number of non-zero components of $\underline{x}$

$L_\infty = \|\underline{x}\|_\infty = \max_i |x_i| =$ largest absolute component of $\underline{x}$



curse of dimensionality: as dimension ↑, variance of distances shrink relative to their mean ⇒ points are equally close

2a)

GD: $\theta_{i+1} = \theta_i - \eta \nabla_\theta \mathcal{L}(\theta)$; mini-batch: "update model every k observations" ; SGD: "mini batch with k = 1" (k = batch size)

momentum: "continue moving in same direction" ; RMS Prop: "small (large) steps for large (small) gradients" ; Adam: "Momentum + RMS Prop"

Adagrad: uses a per-feature learning rate" ; Coordinate descent: "use when in L_1 world"

3a)

$p(\theta|D) = \frac{p(D|\theta)\,p(\theta)}{p(D)}$ (likelihood, prior, posterior, marginal)

MLE: $\arg\max_\theta p(D|\theta)$

MAP: $\arg\max_\theta p(D|\theta)p(\theta)$

→ solve → $\log(\cdot) = 1$, $\frac{\partial}{\partial\theta} = 1\,(=0)$

MLE & MAP both consistent; MLE overfits more than MAP

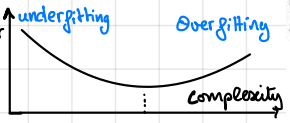
MLE = MAP with uniform prior ; X^TX singular ⇒ no MLE

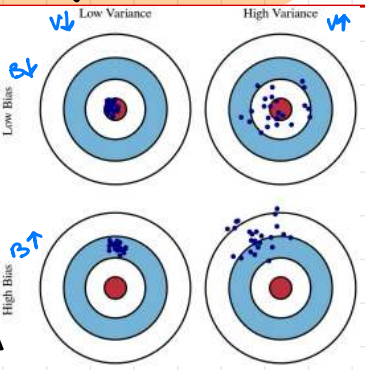
$\hat{w}_{OLS} = \hat{w}_{MLE}$ w/ Gaussian noise, $\hat{w}_{ridge} = \hat{w}_{MAP}$ w/ Gaussian prior on w

$\hat{w}_{MLE} = \frac{\sum x_i y_i}{\sum x_i^2}$; $\hat{w}_{MLE} = (X^TX)^{-1}X^TY$; $\hat{w}_{MAP} = (X^TX + \lambda I)^{-1}X^TY$

simple model: bias ↑ var ↓ ; complex model: bias ↓ var ↑

the simpler (more complex) the model, the less (more) well it's gonna fit the data, but the less (more) it's gonna vary for ≠ datasets (i.e. the more (less) stable it's gonna be)





Test error = Train error + Model complexity = Variance + Bias² + Noise

K-Fold CV: used to pick hyperparams & estimate test error ; if $k = n \Rightarrow$ LOOCV

$\text{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta} - \theta]$; $\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$

adding penalties to OLS: $(\|y - Xw\|_2^2 + \lambda\|w\|_p^p$ { p=2 (ridge) convex, makes w's smaller but never 0 ; p=1 (LASSO) convex, drive some w's to 0 (feature selection) ; p=0 (stepwise) non convex, requires search, but stronger than L_1 }

⇔ MAP with Gaussian (p=2), Laplace (p=1) priors

stepwise regression: look at all remaining features at each step and pick the best to add (~greedy)

streamwise regression: look at one feature at a time, in a fixed order, immediately accept or reject, no revisits (more greedy)

4a)

logistic regression for binary classification, uses the sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$; log odds: $\frac{P(Y=1|x,w)}{P(Y=-1|x,w)} = w^Tx$ → if = 0 ⇒ hyperplane decision bound

Multi-class classification for K classes uses $p(y=k|x,\theta_1,\dots,\theta_k) = \frac{e^{\theta_k^T x}}{\sum_k e^{\theta_k^T x}}$ (softmax function) → maps a vector to a probability distro

↳ predict class as most probable label

RBF: $k\left(\frac{\|x - x_j\|}{c}\right) = \exp\left\{\frac{\|x-x_j\|^2}{c}\right\}$. "Kernel width" c small ⇒ overfit, c big ⇒ underfit

↳ to find kernel centers, do k-means and take cluster centers

if { i) $d < p$ basis vectors ⇒ dimensionality reduction ; ii) $d > p$ ⇒ dimensionality increase/kernelization (make non linear pb linear) ; iii) $d = n$ ⇒ switch to dual representation }

4b)

a kernel function measures the similarity between two points and induces an SPSD matrix $K_{ij} := k(x_i, x_j)$

- kernels assume similarity is the same everywhere
- SVMs: used for classification with hinge loss $= \max(0, 1 - y^* f(x))$ ← Points on correct side of margin don't contribute to hinge loss, so loss "entirely in terms of points inside margin or on wrong side of it"

kernel regression: $\hat{y}(x) = \frac{\sum K(x,x_i)y_i}{\sum K(x,x_i)}$ ↳ non parametric

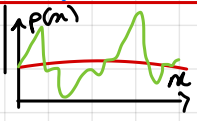
kernel classif: $\hat{y}(x) = \text{sign}(\sum K(x,x_i)y_i)$

5a)

entropy of a rv X ; $H(X) \in [0,\infty)$ but $\in [0,1]$ if X binary

$H(X) = -\sum_{i=1}^{m} p_i \log_2(p_i)$; m = # events E_i and $p_i = P(E_i)$

= "average conditional entropy of Y"



low entropy / high entropy

- two events are equally likely ⇒ $H(X) = 1$
- $H(X) = 0$ ⇒ an event is certain
- better model ⇒ lower $H(Y|X)$

conditional entropy: $H(Y|X) = \sum_{i=1}^{m} P(X=E_i)\,H(Y|X=E_i)$; Information Gain: $IG(Y|X) = H(Y) - H(Y|X)$; cross entropy: $H(P,Q) = -\sum p(x)\log(q(x))$ (Q = approx distro)

KL divergence: $D_{KL}(P\|Q) = \sum p(x_j)\log\frac{p(x_j)}{q(x_j)} = \dots = H(P,Q) - H(P)$ (P = true distro) ≥ 0 ; non-symmetric, measures difference ; and $D_{KL}(P\|Q) = 0 \Leftrightarrow P = Q$

decision tree algo: Recursive partition to maximize IG. if overfit → pruning, max_depth, or other

8b | **ensembles**: combine many weak models into a strong one. **bagging**: draw bootstrap sample with replacement, train weak model
Random Forests: Bagging + random feature selection (m) (⇒ no overfit, no underfit) ↳ repeat for T models
stagewise regression: "sequentially learn the weights α_t assigned to models, never readjust"; **boosting**: ⇒ "give more weight to points we got wrong"
Adaboost: minimizes exponential loss, learns exponentially fast, doesn't overfit; **Gradient(T) Boosting**: generalization of ada for any loss
regularization for GB: η (shrinkage); tree depth; boosting stages, ... reduces variance but also bias unlike RF

8a | **SVD**: $X = UDV^T$ ($n\times p = n\times p\ p\times p\ p\times p$); cols of U and V are left and right singular vectors and diag elems of D (diag) are the singular values of X (not V^T!)
$\|A\|_F = \sqrt{\sum_i \sum_j |a_{ij}|^2} = \sqrt{\text{trace}(CA^TA)} = \sqrt{\sum_i^{\min(m,n)} \sigma_i^2(A)}$
$\therefore X = ZV^T$ ($n\times m = n\times r\ r\times m$) where $Z = UD$ = scores & V = loadings, PC's, PC directions/components
PCA: minimizes distortion $\min_{Z,V} \|X - ZV^T\|_F^2$ ⇔ maximizes variance
$\min_{z,v} \|X - zv^T\|_F^2 + \lambda_1\|z\|_1 + \lambda_2\|v\|_2$ → convex in z|v or v|z
Sparse PCA: introduce L_1 penalty on scores or loadings ⇒ no longer eigenvectors, must solve using alternating GD
Moore-Penrose pseudoinverse of X: $X^+ = VD^{-1}U^T$; **thin SVD**: $X^+ \sim V_k D_k^{-1} U_k^T$ (Cheap to k singular vals/vectors) good approx of inverse!
Power Method: finds PC1 by repeatedly multiplying a (square) random vector by X^TX, causing it to align with the dominant eigenvector of X^TX: PC1

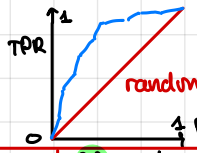
8b | **PCR**: using PCA scores as regression features → 1) do PCA on X get loadings & scores; 2) do OLS using scores: $w = (Z^TZ)^{-1}Z^Ty$
3) compute score $z = V^Tx$ and predict $\hat{y} = w^Tz = w^TV^Tx$. PCA removes colinearity and noise ⇒ regression more stable
semi-supervised PCR: use lots of unlabeled data to find good PCA representation, then regress on the labeled data
PCA for visualization: PCA reduces data to the top 2 PCs to plot and visually inspect patterns, clusters and relationships
• each PC has an eigval λ_i and % variance explained by $PC_i = \frac{\lambda_i}{\sum \lambda_i}$ • U = word embeddings (context-free); V = context embeddings (context sensitive)

8c | **autoencoders**: NNets taking X as input & output and force info through bottleneck/sparsity to learn an embedding
denoising AE: Corrupt input X with noise, reconstruct X cleanly ⇒ learn robust features. Also, **stacked AE**, useful for SL
ICA: given X, find W s.t. $X = SW$ where $s_j \perp\!\!\!\perp$, not just uncorrelated. ICA doesn't have simple SVD solution
↳ reconstruct $X \sim SW^T$ ($S \sim$ PCA scores, $W^T \sim$ loadings). $MI(y_1,\dots,y_m) = KL(p(y_1,\dots,y_m) \| p(y_1)\cdots p(y_m))$
↳ ICA aims for $\perp\!\!\!\perp$ not variance! It does disentanglement, as well as reconstruction

9a | **Prediction** (confusion matrix: Observation)

	P	N
P	TP	FN (II)
N	FP (I)	TN

Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$ misleading under imbalance; Precision = $\frac{TP}{TP+FP}$ = p(yes | predicted yes) measures correctness of positive predictions
Specificity = $\frac{TN}{TN+FP}$ = p(predicted no | no) measures correctness on negatives ↳ 1 - False positive rate
TPR = ↗; Recall = Sensitivity = $\frac{TP}{TP+FN}$ = p(predicted yes | yes) measures catching positives
↳ min proba to call something positive
Precision/Recall trade-off: threshold ↑ ⇒ predicted positives ↓, P↑, R↓; threshold ↓ ⇒ predicted positives ↑, R↑, P↓
↳ P↑ ⇒ R↓ (vice versa), only way to increase both is via a better model, not a threshold
F1 score = $\frac{2PR}{P+R}$ ↳ if both P↑ R↑ ⇒ F1↑
• For medical diagnosis we want Recall ↑. For spam email detection we want specificity ↑



← **ROC**: examines classifier performance across all thresholds
↳ evaluates ranking ability, not final classification
AUC = 1 ⇒ perfect; 0.5 ⇒ random

10a | **kmeans**: 1) pick K cluster centroids at random; 2) assign points to nearest centroid (usually L_2 dist.)
3) set centroid to mean of examples assigned to it; 4) repeat from step 2) until convergence
↳ alternates between assigning points & updating centroids. kmeans minimizes reconstruction error: $J(\mu,r) = \sum_{i=1}^n \sum_{k=1}^K r_{ik}\|\mu_k - x_i\|_2^2$
$r_{ik} = \mathbb{1}(k = \text{argmin}_j \|x_i - \mu_j\|^2)$ ↙ $\mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$ ↙ ⇒ alternating minimization, guaranteed to decrease loss at each step, but not guaranteed to find global optimum (initialization matters!) Hence, run kmeans many times to avoid poor solutions. **Kmeans++**: picks diverse starting points by weighting points by squared distance for chosen centers
kmeans assumes: i) hard clustering (1 cluster/point) ii) clusters are spherical & equal size iii) cluster proba uniform: $\pi_k = \frac{1}{K}$
kmeans is a special case of GMMs. GMMs assumes each cluster is generated from a Gaussian distro with params (π_k, μ_k, Σ_k)
• GMM is a soft clustering algo. K-means = GMM with equal mixing weights $\pi_k = \frac{1}{K}$ and $\Sigma_k = \sigma^2 I$ (shared spherical covariance)
EM algo alternates 1) estimate soft cluster membership $p(z_i = k | x_i)$; 2) update parameters (EM only finds local max)

11a | **GMM**: to generate each point: 1) pick a cluster z w.p. π; 2) $p(x) = \sum_k \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$ where $x \sim \mathcal{N}(\mu_z, \Sigma_z)$
• GMM uses EM because cluster assignment are latent: E: $p(z=k|x)$, M: update μ_k, Σ_k, π_k
• **NB**: assumes words are conditionally $\perp\!\!\!\perp$ given topic (→ "naïve"). If topic labels are known → count, else → EM:
↳ E: infer topic of document; M: Estimate word distribution. Limits: i) 1 topic/doc ii) no word dependencies;
iii) unrealistic linguistically, hence ⇒ **LDA**: 1) for each doc, sample topic proportion $\theta_d \sim \text{Dirichlet}(\alpha)$; 2) for each word choose topic $z_n \sim \text{multinomial}(\theta_d)$; choose word $w_n \sim \text{Multinomial}(\beta_{z_n})$ (≠ w/ NB: Each word can have its own topic)
HMMs: 1) sample init state $s_0 \sim \pi$ 2) repeat: i) Emit observation $x_t \sim p(x_t | s_t)$ ii) transition $s_{t+1} \sim p(s_{t+1} | s_t)$

HMMs are dynamic generative models. For estimation E: estimate hidden state; M: estimate emission and transition probabilities

Belief Nets: DAG, encodes conditional ⫫, factorizes joint: $p(x_1, \dots, x_n) = \prod_i p(x_i \mid \text{Parents}(x_i))$ (key: graph structure = ⫫ assumptions)

LHA: A node is ⫫ of all non-descendants given its parents: $X \perp\!\!\!\perp \text{nondes}_X \mid \text{Parents}_X$

props: ii) $X \perp\!\!\!\perp Y, W \mid Z \Rightarrow X \perp\!\!\!\perp Y \mid Z$; iii) $X \perp\!\!\!\perp Y, W \mid Z \Rightarrow X \perp\!\!\!\perp Y \mid Z, W$ iv) $(X \perp\!\!\!\perp W \mid Y, Z), (X \perp\!\!\!\perp Y \mid Z) \Rightarrow X \perp\!\!\!\perp Y, W \mid Z$

i) $X \perp\!\!\!\perp Y \mid Z \Rightarrow Y \perp\!\!\!\perp X \mid Z$ (btw parents & non-descendants)

- X & Y are d-separated by Z if every path between X and Y is blocked given Z (if ∄ active trail between X and Y) $\Rightarrow X \perp\!\!\!\perp Y \mid Z$

to compute # params in BN: $\sum_{\text{nodes}} 2^{\text{num parents}}$

Belief nets provide a compact representation of joint probability distributions

- Guaranteed to be a consistent specification
- Graphically show conditional independence
- Can be extended to include "action" nodes

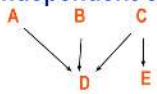
Networks that capture causality tend to be sparser

Estimating belief nets is easy if everything is observable

- But requires serious search if the network structure is not known

How many parameters in the model below?

- A) <12 B) 12-16 C)17-20 D) more than 20
- How does this change if a variable can take on 3 values?



P(A),P(B),P(C): 3 parameters
P(D|A,B,C):8 P(E|C):2 P(F|D,E):4 3+8+2+4=17

What is conditionally independent of what?

- P(D|C) = P(D|C,E)? - yes
- P(D|F) = P(D|F,E)? - no
- P(F|D) = P(F|D,A)? - yes
- P(F|D) = P(F|D,C)? - no

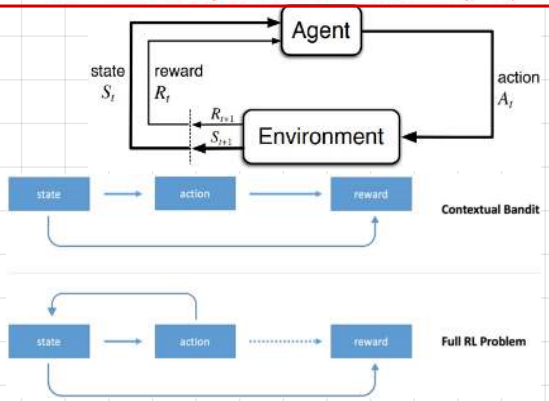
A trail $\{X_1, X_2, \dots, X_k\}$ in the graph (no cycles) is an **active trail** if for each consecutive triplet in the trail:

- $X_{i-1} \rightarrow X_i \rightarrow X_{i+1}$, and X_i is not observed
- $X_{i-1} \leftarrow X_i \leftarrow X_{i+1}$, and X_i is not observed
- $X_{i-1} \leftarrow X_i \rightarrow X_{i+1}$, and X_i is not observed
- $X_{i-1} \rightarrow X_i \leftarrow X_{i+1}$, and X_i is observed or one of its descendants is observed

Variables connected by active trails are not conditionally independent

---

RL is hard: i) never see result of action not taken
ii) never told what best action was
iii) often long sequence of actions before knowing their consequence



types:

- **model-based**: learn full env ($p(s' \mid s,a)$ & $r(s,a)$ = transition model + reward model), then DP or search
  - ↳ MDP, DP, POMDP
  - (ex: chess, grid world with known transitions)
- **model-free**: learn V-values, Q-values or policy (ex: Trading bots / online bidding, atari from pixels)
  - ↳ TD, on/off policy (SARSA, Q-learning), MC

TD(0) update:

$V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$

(discount factor, one-step lookahead; learning rate; immediate reward)

SARSA: (on-policy)

$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma Q(s',a') - Q(s,a))$

uses next action taken

Q-Learning: (off-policy)

$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$

uses optimal future action, learn the optimal policy even while exploring

exploration v/s exploitation strategy:

ε-greedy: P(explore) = ε = 1 - P(exploit)

MDP = $(S, A, p(s' \mid s,a), r(s,a,s'), \gamma)$

↳ value functions: $V_\pi(s) = \mathbb{E}[G_t \mid s_t = s]$; $Q_\pi(s,a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]$ ($\pi \neq \pi$)

↳ $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots$

$\Rightarrow V_\pi(s) = \sum_a \pi(a \mid s) Q_\pi(s,a)$

Bellman's Eq

i) $V_\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a)[r + \gamma V_\pi(s')]$

ii) $V^*(s) = \max_a \sum_{s'} p(s' \mid s,a)[r + \gamma V^*(s')]$

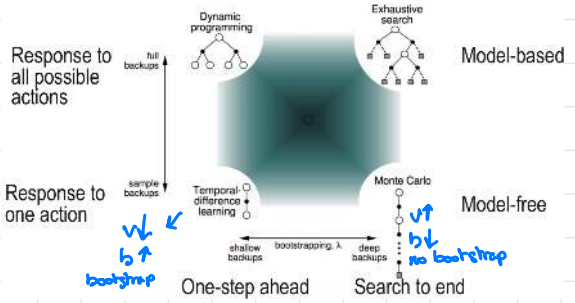
iii) $Q^*(s,a) = \sum_{s'} p(s' \mid s,a)[r + \gamma \max_{a'} Q^*(s',a')]$

policy iteration: (1) compute V_π using ↗ (policy eval); (2) $\pi'(s) = \text{argmax}_a Q_\pi(s,a)$ (policy improvement)

MC RL: V(s) = average of actual returns from episodes

↳ "on policy"

Overview of RL Strategies



Take-aways (1)

- **Superhuman game playing using DQN**
  - interestingly, superhuman play still requires search
  - current RL does not generalize well to new games
- **Many combos of learning policy, models, V, Q-functions**
  - Separate policy selection from policy evaluation
- **Sometimes use "behavioral cloning" to initialize**
- **Often regularize**
  - To stabilize learning
  - To stay closer to previous policies or human play
- **Increasingly use a more abstract state and action space**

Take-aways (2)

Policy gradients work well for alignment
RLHF uses RL to propagate reward at end of the response (estimated by a model) back through the tokens in the response
Lots of tricks to support convergence: clipping, regularization, use of references
Today's methods: on policy, model free
Alternate: DPO directly optimizes the reward difference between a pair of responses

What you should know

Active learning

- Uncertainty sampling — Sample where models is uncertain
- Query by committee — Sample where model disagree
- Information-based loss functions — Maximize information gain (max KL dist $\Leftrightarrow$ max $\mathbb{E}[IG]$)

Optimal experimental design

- A-optimal design — Minimize trace= $\sum \lambda_i$ of $(X^TX)^{-1}$
- D-optimal design — Minimize det= $\prod \lambda_i$ of $(X^TX)^{-1}$
- E-optimal design — Minimize largest eigenvalue $\max \lambda_i$ of $(X^TX)^{-1}$
- *All minimize the variance in estimate of w*

Response surface methods — Sequential experiments for optimization

1. $X = UDV^T$; eigvectors of X^TX = cols of V. eigvals of X^TX = squares of singular values of X. 2. PCs are orthogonal 3. PC scores of X are columns of $Z = UD$ 4. If X is mean-centered, the PCA finds eigvectors of X^TX, right singular vectors of X and projection directions of max covariance of X 5. Scores $Z = XV = UD$ tell you where each sample lies along each PC; loading V tell you which feature contribute to each PC 6. Autoencoders generalize PCA or ICA; and PCA = linear autoencoder with L_2 loss 7. PCA finds direction of max variance, ICA finds direction where data is most non-Gaussian / statistically indep 8. AEs learn weights to minimize reconstruction error. 9. Real-world data often lies near a low-dimensional manifold, AEs (and PCA) try to learn coords on this manifold 10. Stacking AEs gives embeddings to use in SL 11. It is recommended to mean-center X before running PCA on it. 12. The distributional similarity hypothesis for word embeddings states: "words occurring in similar contexts tend to have similar meanings" 13. Relative to PCA, ICA aims to find PCs that are as statistically indep. as possible 14. the non-zero eigenvalues of AA^T and A^TA are the same 15. the left (right) singular vectors of a matrix A are the eigenvectors of AA^T (A^TA) 16. standardizing = mean-centering + ÷ by std

17. mean encoding: replace each category with mean of target variable for that category • Power methods for estimating eigenvectors are attractive because they guarantee to find a global optimum in reconstruction error when used in PCA

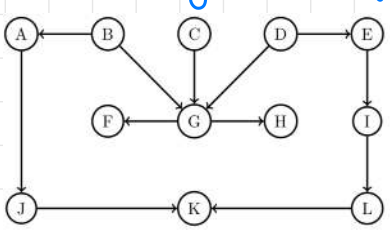
2018

1. (MAYBE) best way to deal with missing data is to replace it by 0 and add an extra column indicating wether data is missing or not. 2. Q-learning has higher bias and lower variance than MC methods 3. doing PCA on a standardized dataset doesn't mean the scores will have variance = 1! 4. Logistic reg gives a globally optimal solution to its loss function (binary cross entropy is convex), unlike NNets or k-means. 5. In a penalized model, the less data we have, the more the overfitting risk, so the larger the regularization penalty selected by cross validation. 6. degree d polynomial kernel SVM is more computationally expensive than linear kernel SVM, which is as expensive as logistic regression 7. A single perceptron update does not guarantee fixing this misclassification of the very point that triggered the update 8. In Adaboost for binary classification, training error is guaranteed to approach zero as the number of iteration $\to \infty$, and it should ideally use an underfit model as the "weak learner" 9. For GMMs, complexity ↑ as # Gaussians ↑, not the case for KNN with K↑, nor NNets with L_2 penalty coef ↑

2018

• hinge loss is convex! exponential loss too! 0-1 loss is non convex! 2. Removal of a support vector doesn't always change the SVM decision boundary. 3. In LR, looking at the largest regression weights in absolute value is NOT a reliable way to determine which features are most influential, UNLESS ALL FEATURES ARE STANDARDIZED! 4. CNNs don't work well on high dimensional data like medical diagnosis from health records (with features like age, weight, temp, ...) 5. the statement: "Vanilla RNNs, unlike HMMs, don't forget things exponentially quickly" is FALSE! • AEs don't ALWAYS reconstruct to a smaller dim ex: "Overcomplete AEs" • "LMS does SGD in a NLL"

"conditionning on desc of collider opens it" ← ex: G



$C \perp\!\!\!\perp D \mid F$ F
$D \perp\!\!\!\perp I \mid E, F, K$ F
$C \perp\!\!\!\perp J \mid A, F, L$ T
$F \perp\!\!\!\perp L \mid G$ T
$\neg(G \perp\!\!\!\perp E \mid D, K)$ T
I d-separates E and L T

scale inv: Linear regression (OLS), Decision trees, L_0, hard SVMs?
non scale inv: Ridge (unless features are std), L_1, k-NN, PCA, SVMs
coord-based: Gradient tree boosting
not coord-based: NNs, Linear / logistic regression